

Process & Decision Documentation

Entry Header

Name: Lee Golan, Katherine Chen, Scarlet Sun, Giang Tran, Keyan Virani, Emily Huang

Role(s) & Primary responsibility for this work

Name	Preferred Contact Method	Role	Shadow Role
Katherine Chen	Instagram, Discord	Dev 1	Game Design
Giang Tran	Instagram, Text	Dev 2	Game Design
Haojia/Scarlet Sun	Instagram, Discord	Artist	Story/level design, coding
Emily Huang	Instagram, Discord	Sound + UI	Game Design
Keyan Virani	Instagram, Discord, text	Lead Game Design + Flow/Levels	Art
Lee Golan	Instagram, text	Dev 3	Game design + story
All		Story/Plot building	

Goal of Work Session

Tools, Resources, or Inputs Used

- Claude Sonnet 4.5 and Opus 4.5 for code structuring and prompt engineering
- ChatPRD for creating game specifications
- Pixabay and Garage Band for sound effects and BGM
- Week 5 In-Class Game Example for foundational code base

Process Overview Visualization (Group Work Only)

1. Group Members

List the full names of all group members and the best way to contact each person (for example, Discord, email). Do not include personal information such as usernames or phone numbers on this document. You may share them among each other if you are comfortable. Each member must choose a role and shadow role.

Name	Preferred Contact Method	Role	Shadow Role
Katherine Chen	Instagram, Discord	Dev 1	Game Design
Giang Tran	Instagram, Text	Dev 2	Game Design
Haojia/Scarlet Sun	Instagram, Discord	Artist	Story/level design, coding
Emily Huang	Instagram, Discord	Sound + UI	Game Design
Keyan Virani	Instagram, Discord, text	Lead Game Design + Flow/Levels	Art
Lee Golan	Instagram, text	Dev 3	Game design + story
All		Story/Plot building	

TIMELINE

Date	Deliverables	Person responsible
Finish Level 2 Changes (platformers)	Saturday, 28th March	Lee
Finish Level 3 Changes (platformers)	Sunday, 29th March	Giang
Implement popup interactions & polish	Monday, 30th March	Lee, Kat, Giang
Visuals	L2: Mid-Saturday L3: Sunday	Emily (background L2) Scarlet (sprite, background & pop up - physical sticky notes for level 3) Keyan (inside of the computer, L3)
April 2	Final game, polished for in-class playtesting	

Playtesting 3:

MVP:

- Add two platform states:
 - Moving platforms
 - Invisible platforms

Can Try:

- Popups shake rather than static in second level
- Be able to jump on platforms on third level

Scarlets Idea:

- Have a mix between realism and online
 - Hand cursor
 - Sticky note popups
 - Flipping paper audio

Third round:

- Cover the screen, smaller pop-ups
- New sprites (picture of the hand holding a cursor) , but decide to go back to internet cursor because it felt more nature

More polished, debugged most things

Project/Assignment Decisions

A1 – A3 (Group Work).

Decision 1: Platform Mechanics – Introducing Unpredictability

- **What changed:** We added two new platform states: moving platforms and invisible platforms as part of our MVP.
- **Why the decision was made:** These mechanics reinforced the core feeling of instability and lack of control. By making platforms unreliable, players experience a constant need to adapt, mirroring the unpredictability we aimed to express through gameplay.

Decision 2: Popup Behavior – From Static to Dynamic

- **What changed:** We explored making popups shake instead of remaining static, particularly in the second level.
- **Why the decision was made:** Static elements felt too controlled and predictable. Introducing motion added tension and distraction, better reflecting the intrusive and disruptive nature of involuntary experiences.

Decision 3: Expanding Player Interaction – Platform Jumping

- **What changed:** We considered allowing players to jump onto platforms in the third level.
- **Why the decision was made:** Adding this interaction increased player agency while still maintaining challenge. It created a more engaging experience and allowed for more layered navigation without losing the core emotional tension.

Decision 5: Visual Direction – Realism vs Digital Aesthetic

- **What changed:** We experimented with combining realistic elements (hand cursor, sticky notes, paper sounds) with a digital interface style.
- **Why the decision was made:** This hybrid approach was meant to blur the boundary between physical and digital experiences, making the interaction feel more tactile and immersive. It also aligned with the idea of intrusive elements appearing in familiar, everyday contexts.

Decision 6: Cursor Design – Returning to Simplicity

- **What changed:** We tested a new sprite (a hand holding a cursor) but ultimately returned to a standard internet cursor.
- **Why the decision was made:** The more complex visual felt unnatural and distracting. The default cursor was more intuitive and familiar, allowing players to focus on the experience rather than the interface itself.

Decision 7: Popup Density – Increasing Visual Overwhelm

- **What changed:** In later iterations, we introduced more frequent and smaller popups that could cover larger portions of the screen.
- **Why the decision was made:** Increasing the density of popups amplified the feeling of overwhelm and loss of control. This supported our goal of creating an emotionally intense and immersive experience.

Role-Based Process Evidence

For group assignments (A1 – A3), this section is completed collectively, with clearly attributed role-based entries. For the group, your entries should correspond to elements shown in your process overview visualization such as stages, iterations, or decision points.

Lee:

Editing level 2.json

add in all the different platform types into level 2

Looking at level 1, it uses `normal`, `slippery`, `slow`, and `falling` types. Let me redistribute those across level 2's platforms in a way that feels balanced — more forgiving early on, getting harder toward the top.

Ran a command, created a file, read a file >

Here's how the types are distributed across the level's three sections:

Bottom third (y 8100-5600) — easing in: mostly `normal`, one `slow`, one `slippery`, one `falling` to introduce mechanics gently.

Middle third (y 5600-3600) — ramping up: even mix of all four types, `falling` platforms start appearing more regularly.

Top third (y 3600-200) — hardest section: frequent `slippery` and `falling`, with `slow` mixed in to break up the rhythm, mirroring how level 1 gets toughest near the finish.

</>

Level2
Code - JSON

Download

Reply...

+ Sonnet 4.6

Claude is AI and can make mistakes. Please double-check responses.

Editing level 2.json

how would i change the background image for level 2

In `sketch.js`, `bgImg` is a single image loaded once and used for all levels. To have a different background per level you'd need to:

1. Add a second image variable and load it in `preload`:

```
javascript
let bgImg2 = null;
```

```
javascript
bgImg2 = loadImage(
  "assets/images/background2.png",
  (img) => { bgImg2 = img; },
  () => { bgImg2 = null; },
);
```

2. In the draw loop, swap which image gets passed to `drawWorld` based on the level:

Reply...

+ Sonnet 4.6

Claude is AI and can make mistakes. Please double-check responses.

Editing level 2.json

make it so if i click "8" on my keyboard I can skip level 1 and immediately start at level 2

Find this block in your `keyPressed` function:

```
javascript
if (gameState === "win" && keyCode === ENTER) {
  gameState = "playing";
  if (levelIndex + 1 === 1) hearts = 4;
  loadLevel(levelIndex + 1);
}
```

Add one line after it:

```
javascript
if (key === "8") {
  hearts = 4;
  loadLevel(1);
}
```

Reply...

+ Sonnet 4.6

Claude is AI and can make mistakes. Please double-check responses.

Editing level 2.json

how do i add a different background music track to level 2

In `sketch.js` make these three changes:

1. Add `sndBgMusic2` variable near the top where the sound assets are declared:

Find:

javascript
// Sound assets
let sndJump, sndBgMusic, sndGameOver, sndRespawn, sndPopup;

Replace with:

javascript
// Sound assets
let sndJump, sndBgMusic, sndBgMusic2, sndGameOver, sndRespawn, sndPopup;

↓

Reply...

+

Sonnet 4.6

Claude is AI and can make mistakes. Please double-check responses.

Editing level 2.json

make it so that when you get a "game over" in a level, you respawn in the same level you died in instead of restarting the entire game from level 1

In your `keyPressed` function, find:

javascript
if (gameState === "gameover" && (key === "r" || key === "R")) {
 gameState = "playing";
 hearts = 4;
 loadLevel(0);
}

Replace with:

javascript
if (gameState === "gameover" && (key === "r" || key === "R")) {
 gameState = "playing";
 hearts = 4;
 loadLevel(1);
}

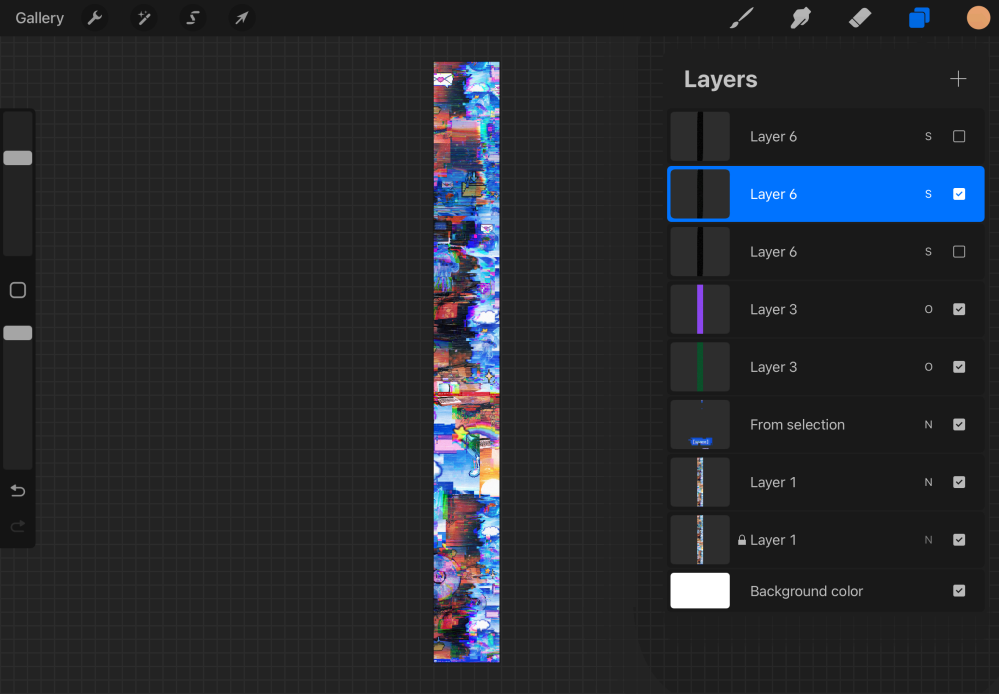
↓

Reply...

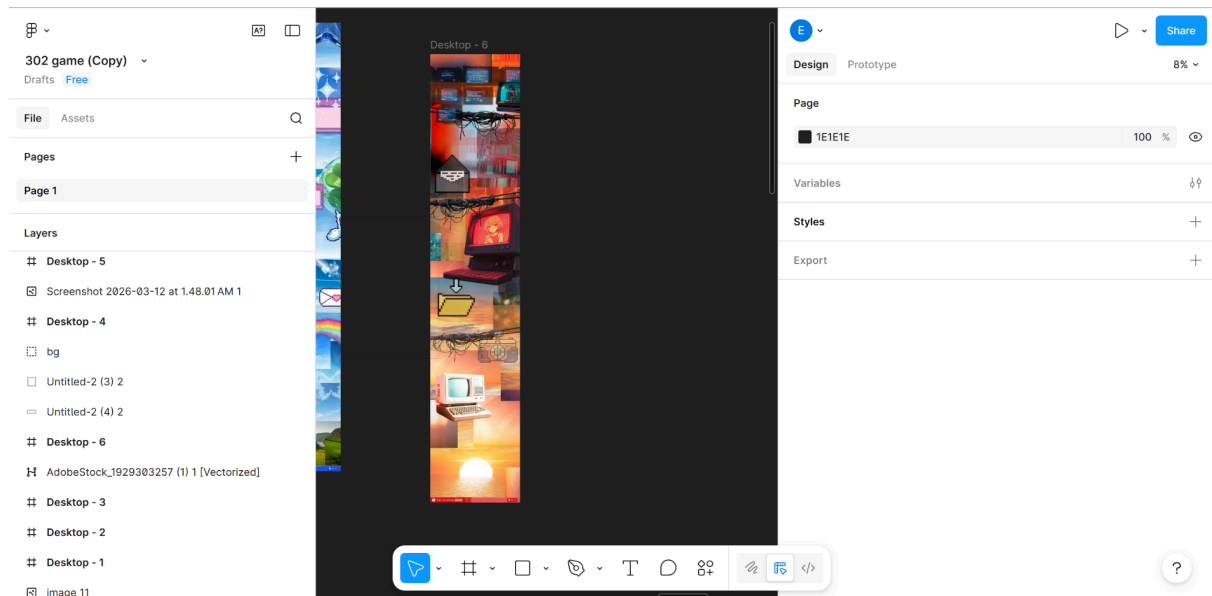
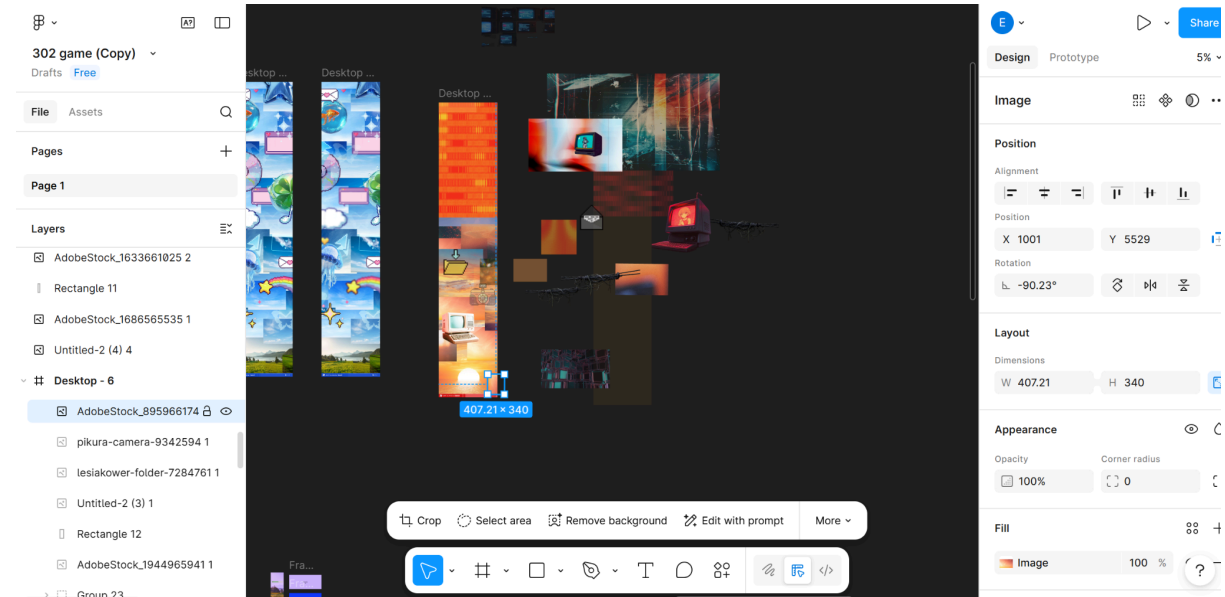
+

Sonnet 4.6

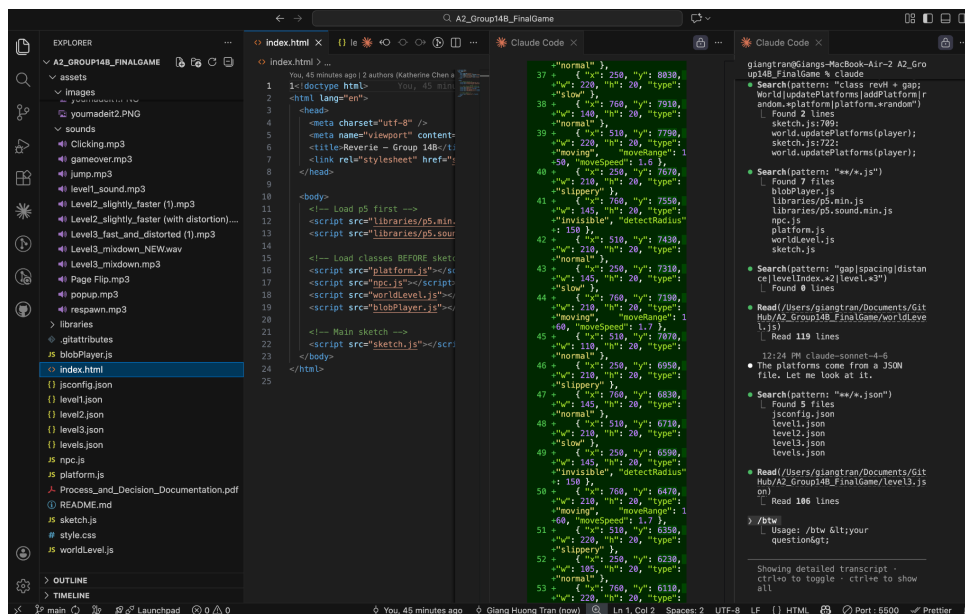
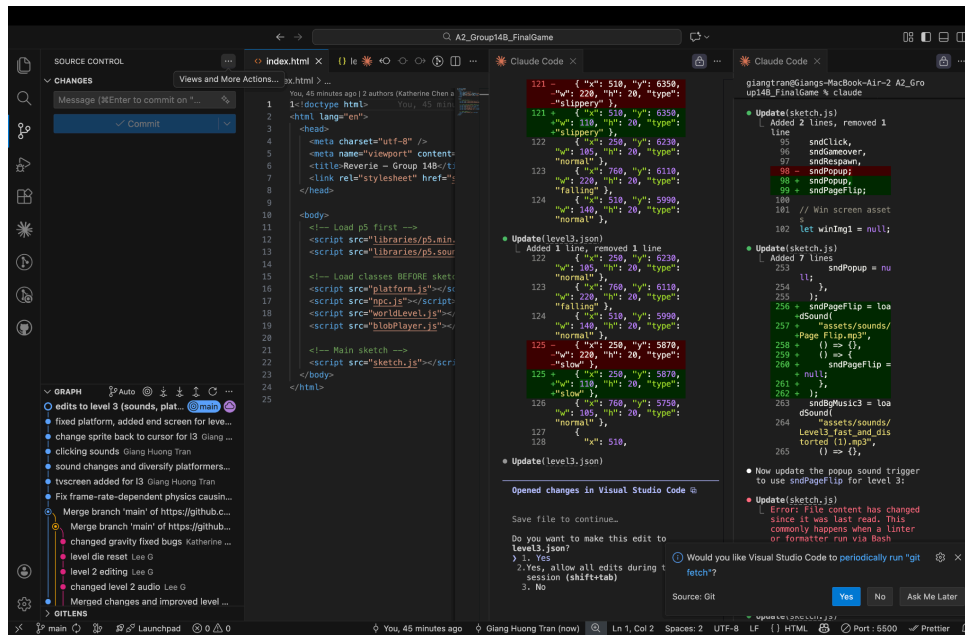
Claude is AI and can make mistakes. Please double-check responses.



Emily

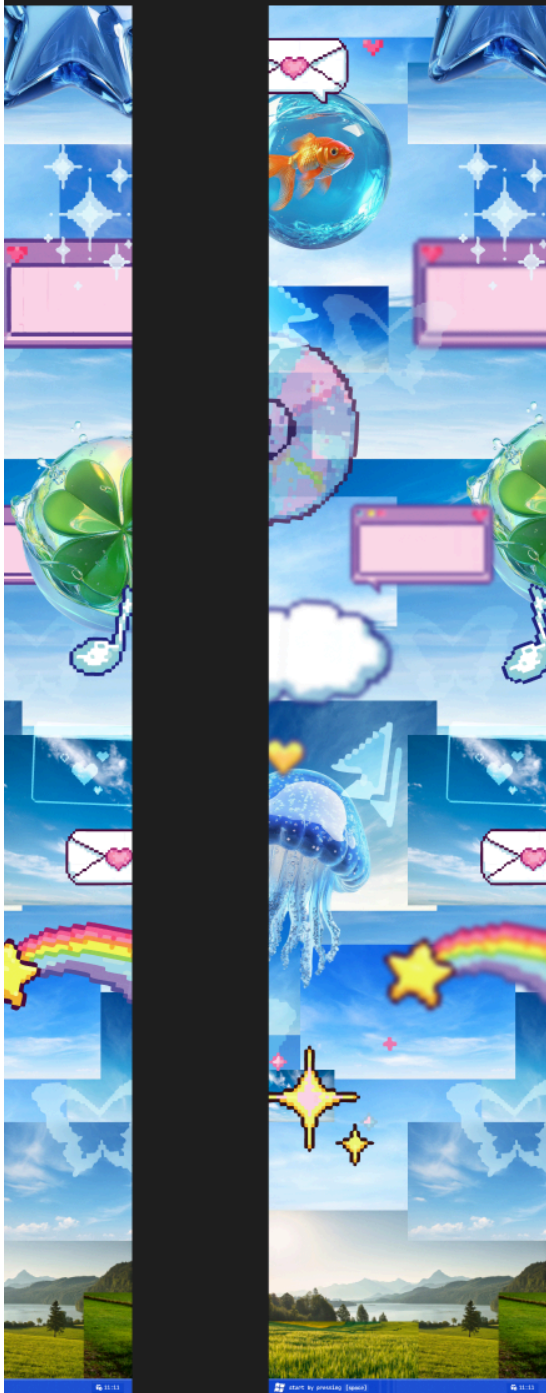


Giang



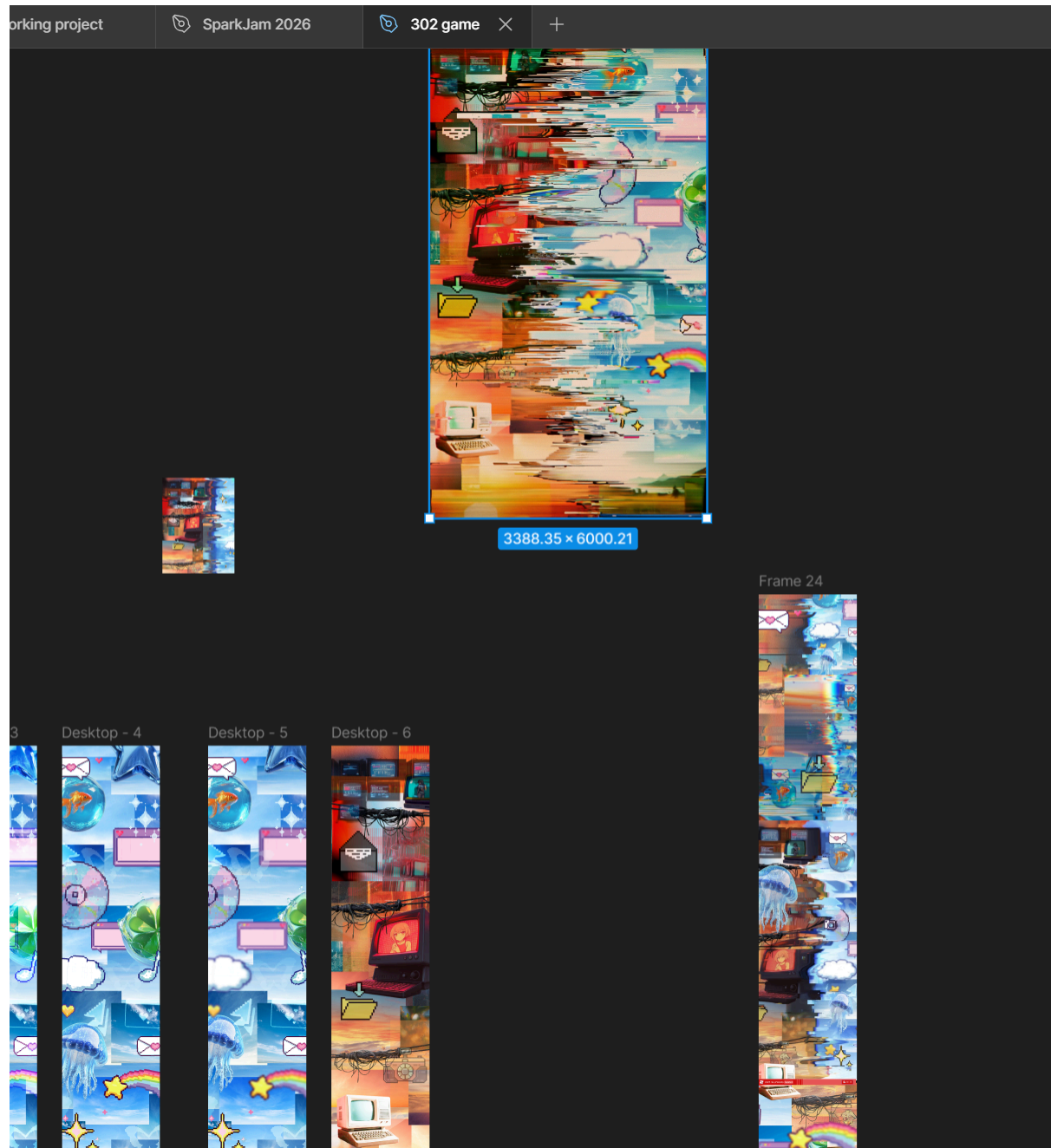
Keyan Virani

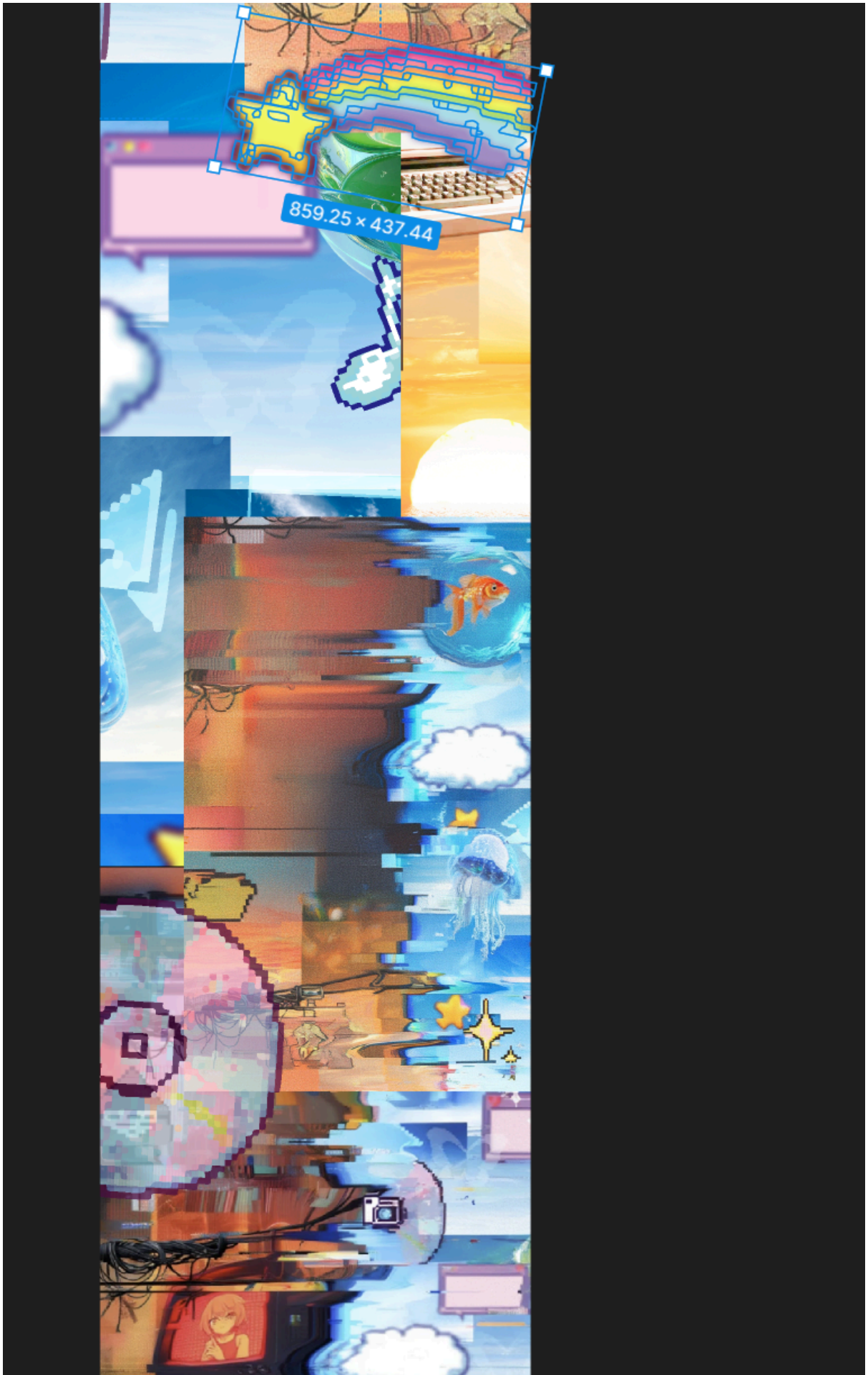
Desktop - 5

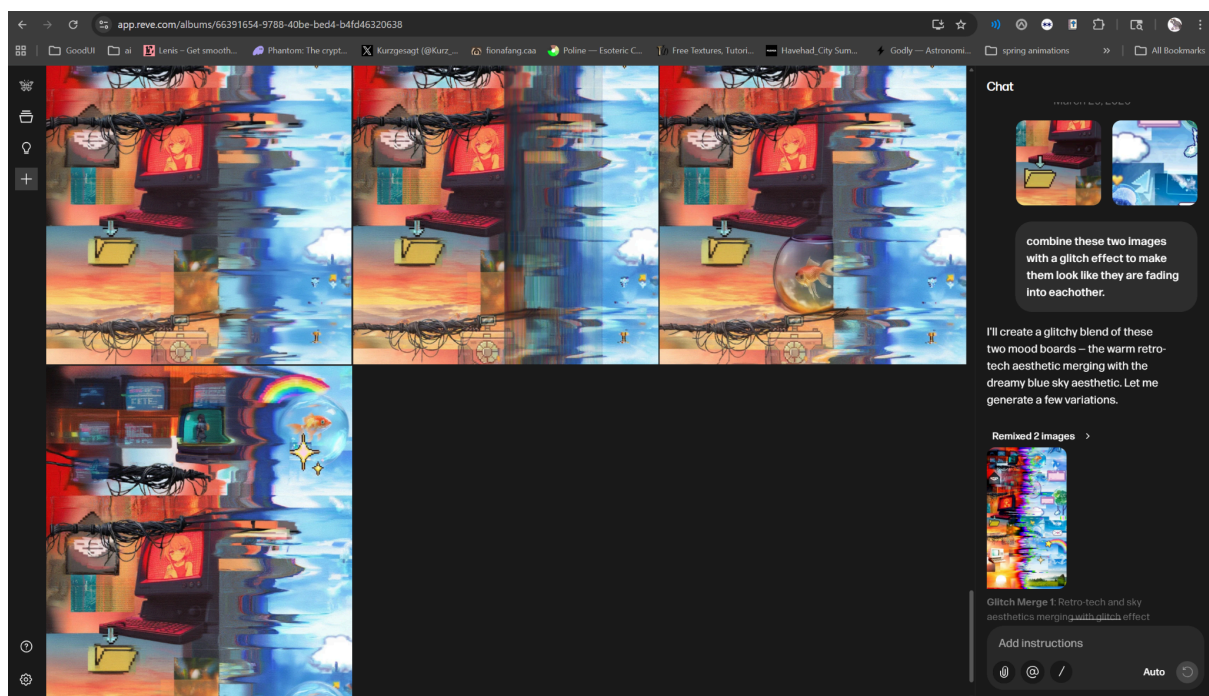


Desktop - 6











Chat

Suggested responses ▾

Generate separate top, middle, and bottom sections to stitch

I'll generate three separate sections that you can stitch together into one tall 1200×10800 image – each section will share the same aesthetic and flow seamlessly into the next.

Remixed 3 images >



Top Section: Top section – warm retro-tech with VHS texture, clear edges

GenAI Documentation

If GenAI was used (keep each response as brief as possible):

Date Used:

- March 27–28: Built and refined Level 2 using Claude Sonnet 4.5. Added train-themed platforms in JSON and adjusted spacing and collision for readability.
- March 28–29: Updated Level 3 using Claude Sonnet 4.5. Reworked spacing (150 → 120), reduced platform sizes, fixed jumpV (-14 → -16), and recalculated level height/checkpoints.
- March 30: Implemented and polished popup system using Claude Sonnet 4.5. Added Level 3 Page Flip sound and tested stress-based popup triggers.
- Mar 30: Used Reve to generate multiple glitch effects of our created backgrounds

Tool Disclosure:

- Claude Sonnet 4.5
- Reve

Purpose of Use: We used GenAI tools like ChatGPT and Claude to help us iterate directly based on issues we found during Playtesting 3, especially for Level 3 difficulty and Level 2 layout. Similar to our previous assignment, we were not using AI to decide our concept, but more to help us quickly test changes and fix problems that came up during development. A lot of our prompts were very specific and based on what we experienced while playing the game, such as adjusting platform spacing, changing jump height, and modifying platform sizes. For example, we asked GenAI to “make the platforms a bit further away for Level 3” to increase difficulty, and then later asked it to “bring them a little bit closer because it’s really hard to jump.” We also used it to increase jump height when we realized the player could not reach certain platforms, and to reduce platform widths to about half to make movement more precise. We also used GenAI to change the popup sound in Level 3 to a page flip sound to better match the sticky note and paper visuals we introduced. For Level 2, we prompted GenAI to generate a new JSON layout using train-themed platforms like benches and seats, based on the structure of Level 1, which we later adjusted ourselves.

Reve generated glitch effect that would be more difficult to create by hand

Summary of Interaction: Our interaction with GenAI was highly iterative and constant prompts to help fix some gameplay issues. We provided specific instructions such as “make the platforms further apart to increase difficulty,” and “increase jump height because the player cannot reach the first platform.” In response, GenAI edited our JSON files by recalculating level height, adjusting platform y-values, and modifying

jumpV values. It also explained why certain problems were occurring, such as identifying that the gap between the starting position and the first platform exceeded the player's maximum jump height based on the current physics. In another session, we prompted GenAI to change Level 3 popup sounds to a page flip effect, and it helped implement the logic for using different sounds depending on the level index.

Prompted reve and chose based on outputs to refine further based on desired result. Reve could not generate long enough images so i generated multiple in similar styles and stitched together manually

Human Decision Point(s): Even though GenAI responded directly to our prompts, we still made all the final decisions based on testing and how the game actually felt. For example when we asked to increase jump height, we accepted the change because it solved the issue of unreachable platforms, but we still tested it to make sure it didn't remove the challenge completely. When we reduced platform widths, the AI applied the change, but this decision came from us wanting to increase precision and tension in the gameplay. For the popup sound change, GenAI implemented the logic, but we chose the page flip sound because it matched the sticky note and paper direction we were developing. In Level 2, although GenAI generated the layout, we guided the theme and made sure it fit our overall aesthetic rather than just using it exactly as given.

Reve split the bgs down the middle. Manually stuck images together using figma, added textures, and added elements to both sides

Integrity & Verification Note:

- All GenAI outputs were tested directly in the game before being finalized
- We played through Level 3 multiple times after spacing and jump changes to confirm jumps were possible
- Platform width changes were tested to ensure they increased difficulty without making gameplay unfair
- Verified that popup sound changes (page flip) worked correctly in Level 3 and matched visuals
- Checked that Level 2 generated layout flowed properly and made sense during gameplay
- Rejected or adjusted any AI output that made the game frustrating or unplayable
- Constantly compared changes against our intended gameplay experience, not just technical correctness

Scope of GenAI Use: GenAI was mainly used to respond to specific prompts related to gameplay issues, and generating initial level layouts. It helped speed up the iteration process by quickly applying changes to our code and JSON files. However most of the outputs from GenAI were adjusted after testing, so they were not used exactly as generated.

Limitations or Misfires:

- When we asked GenAI to increase difficulty by spacing platforms further apart, it did not consider the player's jump limits, which made some sections basically impossible to complete
- It did not always understand the relationship between gravity and jump height unless we clearly explained it, so some outputs needed multiple prompts to fix
- Some generated layouts were technically correct but didn't feel good or fun to play, so we had to adjust them ourselves after testing
- GenAI tends to focus more on logic rather than player experience, so it wouldn't always catch when something felt frustrating or unfair
- If our prompts were too vague, the outputs became very generic or not really useful for our certain creative vision of the game
- Sometimes it followed instructions too literally (like spacing platforms) without thinking about overall gameplay balance

Summary of Process (Human + Tool)

- After Playtesting 3, we identified key gameplay issues such as platform spacing being inconsistent in difficulty and certain jumps not being reachable, so we converted these problems into specific GenAI prompts like adjusting platform distance or increasing jump height to improve playability
- GenAI responded by modifying our JSON level files and adjusting key variables such as y positions, level height, and jumpV values to directly change platform placement and player movement behaviour
- We implemented these generated changes into the game and tested them in real gameplay conditions to evaluate whether the adjustments actually improved difficulty balance and overall level feel
- This process required multiple iterations of prompting, testing, and refinement especially in Level 3; and we also used GenAI for smaller tasks such as popup sound adjustments and generating the Level 2 layout structure

Decision Points & Trade-offs

Platform Design

Level 3's original spacing made several jumps feel harsher than intended. I tightened the vertical gaps and recalculated the level height so the climb stays challenging without turning into guesswork. Platform widths follow a steady pattern: wider early on, narrower near the top to add pressure. Using JSON made these adjustments quick updating y-values was enough to rebalance the whole level.

Camera & Physics

The camera keeps the player around 60% from the top, which gives more room to see upcoming platforms. Physics values stay consistent across all levels so difficulty comes from layout, not movement changes. AABB collision was kept because it's predictable and lightweight, which helps during sections with many moving platforms.

Stress & Popups

The stress meter stays hidden to keep the metaphor intact. At max stress, eight popups appear enough to crowd the screen without blocking platforms. Spawn ranges avoid the edges so the layout stays readable. For Level 3, I switched the popup sound to a Page Flip effect to match the level's more clinical theme.

Visual & Audio

Each level uses a different palette to support the emotional progression: warm in Level 1, cooler in Level 2, and desaturated in Level 3. The Level 3 track is intentionally fast and distorted to add pressure without becoming distracting.

Technical Considerations

Storing platforms in JSON keeps the game modular. When Level 3 spacing needed changes, all platforms could be updated at once instead of editing

Verification & Judgement

- Created debug mode to display stress values on screen and confirm that velocity-based stress increases were working correctly. Tested popup behaviour at different stress levels (30, 60, 85, 100) to check that the correct number of popups triggered and that Level 3 correctly switched to the Page Flip sound instead of the default popup audio.
- In Level 3, early testing showed the first jump was impossible due to insufficient jump height, so jumpV was increased (-14 → -16). This fixed progression without affecting Level 1 or Level 2 movement feel.
- Level 3 spacing was adjusted from 150 → 120 units after testing showed jumps were too punishing and unclear. This made the level more readable while still keeping difficulty through tighter timing. A second adjustment reduced several platform widths from ~210–220 to ~105–110, which increased precision requirements but kept the level fair overall.

- Level 2 was tested through its train-themed platform design, making sure objects like benches, seats, and tables were still readable as platforms in-game. Even with different shapes and sizes, players were still able to recognise them quickly, which confirmed that the visual language was clear without needing labels.
- Collision testing included intentionally trying to break platforms through corner jumps, fast drops, and spam jumping. No major issues were found and performance stayed stable at 60fps even in Level 3 with more platforms.

Limitations, Dead Ends, or Open Questions

- Levels are fully hand-built using JSON, which made Level 3 iteration (spacing, height, checkpoints) quick to adjust. However, this approach does not scale well, and expanding the game further would require a more automated or procedural system.
- There were balancing issues where platform spacing, jump strength, and stress effects sometimes stacked together and created difficulty spikes. For example, Level 3 initially had jumps that were impossible until jumpV and spacing were adjusted together, showing that these systems are tightly dependent.
- Hearts currently persist across levels, but this creates unclear difficulty pacing between Level 2 and Level 3. It may need to be reset per level or redesigned depending on how punishing progression is meant to feel.
- AI feedback (ChatGPT and Claude) was useful for speeding up iteration, especially with JSON edits and debugging spacing issues. However, suggestions like fully random popups or linear stress scaling were not used because the design relies on threshold-based stress changes rather than continuous scaling.

•

Appendix

for the pop ups of level 3
specifically, change their
sounds
to Page Flip

searched for 4 patterns, read 1
file (ctrl+o to expand)

There's already a Page Flip.mp3 in
the assets. Now I'll load it and
use it for level 3 popups
specifically.

Read 1 file (ctrl+o to expand)

Update(sketch.js)
└ Added 2 lines, removed 1
line
95 sndClick,
96 sndGameOver,
97 sndRespawn,
98 - sndPopup;
98 + sndPopup,
99 + sndPageFlip;
100
101 // Win screen asset
s
102 let winImg1 = null;

Update(sketch.js)
└ Added 7 lines
253 sndPopup = nu
ll;
254 },
255);
256 + sndPageFlip = loa
+dSound(
257 + "assets/sounds/
+Page Flip.mp3",
258 + () => {},
259 + () => {
260 + sndPageFlip =
+ null;
261 + },
262 +);
263 sndBgMusic3 = loa
dSound(
264 "assets/sounds/
Level3_fast_and_dis

```
torted (1).mp3",  
265    () => {},
```

- Now update the popup sound trigger to use sndPageFlip for level 3:

- Update(sketch.js)
 - Error: File content has changed since it was last read. This commonly happens when a linter or formatter run via Bash rewrites the file. Call Read on this file to refresh, then retry the edit.

Searched for 1 pattern, read 1 file
(ctrl+o to expand)

- Update(sketch.js)
 - Added 7 lines, removed 1 line
 - 523 // Kick off p
op animation — star
t at small scale
 - 524 if (popupScales[nextSlot] === 0)
popupScales[nextSlot] = 0.01;
 - 525 visibleSlotCount++;
 - 526 - if (sndPopup)
- {
 - 526 + if (levelIndex === 2) {
 - 527 + if (sndPageFlip) {
 - 528 + sndPageFlip
+ip.stop();
 - 529 + sndPageFlip
+ip.setVolume(0.5);
 - 530 + sndPageFlip
+ip.play();
 - 531 + }
 - 532 + } else if (sndPopup) {
 - 533 sndPopup.stop();

	<pre> 534 sndPopup.se tVolume(0.5); 535 sndPopup.pl ay(); </pre> 🔴 Done. Here's what was changed: 1. Declared sndPageFlip in the sound assets variables (line 98). 2. Loaded "assets/sounds/Page Flip.mp3" in preload(). 3. Updated the popup spawn sound logic — when levelIndex === 2 (level 3), it plays sndPageFlip instead of sndPopup. Levels 1 and 2 <pre> cosketch.js:622:he original popup play .update(world.platforms, dt); * esketch.js:640: const fini hPlatfo m = › rworld.platforms.find((p) =>s a p.typee=== "finish");ch other, n sketch.js:642: t finishPlatform && icult sketch.js:644: player.y + plhyer.r > finishPlatform.y + f finishPlatform.ha+ 10 sketch.js:651: 🔴 ThefinishPlatforme&&rom a JSON sketch.js:657: at it. player.x > finishPlatform.x && sketch.js:658: player.x < finishPlatform.x + finishPlatform.w && sketch.js:659: player.y </pre>
for level 3, make the platforms a bit further away from each other, ranging from 2px to 7px. i want it to be a little bit more difficult	🔴 Search(pattern: "platform Platform") └ Found 24 lines sketch.js:2:Week 4 — Example 5: Example 5: Blob Platformer (JSON + Classes) <pre> sketch.js:558: let candidates = world.platforms.filter(</pre>


```

    sketch.js:566:    candidates
    = world.platforms.filter(
    sketch.js:574:    let
    bestPlatform = null;
    sketch.js:581:
    bestPlatform = p;
    sketch.js:585:    if
    (bestPlatform) {
    sketch.js:586:        player.x =
    bestPlatform.x + bestPlatform.w
    / 2;
    sketch.js:587:        player.y =
    bestPlatform.y - player.r - 1;
    sketch.js:622:
    player.update(world.platforms,
    dt);
    sketch.js:640:    const
    finishPlatform =
    world.platforms.find((p) =>
    p.type === "finish");
    sketch.js:642:
    finishPlatform &&
    sketch.js:644:    player.y +
    player.r > finishPlatform.y +
    finishPlatform.h + 10
    sketch.js:651:
    finishPlatform &&
    sketch.js:657:    player.x >
    finishPlatform.x &&
    sketch.js:658:    player.x <
    finishPlatform.x +
    finishPlatform.w &&
    sketch.js:659:    player.y +
    player.r >= finishPlatform.y -
    2 &&
    sketch.js:660:    player.y +
    player.r <= finishPlatform.y +
    finishPlatform.h + 10;
    sketch.js:709:
    world.updatePlatforms(player);
    sketch.js:722:
    world.updatePlatforms(player);
    sketch.js:1094:    const
    finishPlatform =
    sketch.js:1095:    world &&
    world.platforms.find((p) =>
    p.type === "finish");

```

```
sketch.js:1096: if  
(finishPlatform) {
```

```
🔍 Search(pattern: 7:    player.x =  
"generatePlatforms|buildWorld|World  
Builder|gapX|gap|spacing")
```

```
└ Found 16 lines:    player.y =  
sketch.js:386: const spacing =  
50;  
sketch.js:396: let pillW =  
total * spacing + padX * 2 -  
(spacing - size);  
sketch.js:401: const cx =  
startX + i * spacing + size / 2;  
sketch.js:776: let gap = 24;  
sketch.js:782: let totalH = h1  
+ h2 + h3 + gap * 2;  
sketch.js:798: if (winImg2)  
image(winImg2, cx, startY + h1 +  
gap, imgW, h2);  
sketch.js:804: text("Press  
ENTER to continue", width / 2,  
startY + h1 + gap + h2 / 2);  
sketch.js:807: if (winImg3)  
image(winImg3, cx, startY + h1 +  
h2 + gap * 2, imgW, h3);  
sketch.js:816:    startY + h1  
+ h2 + gap * 2 + h3 / 2,  
sketch.js:945: // Sizes —  
reverie is now ~2x bigger, tight  
gap to button  
sketch.js:958: let gap = 32; //  
tight gap  
sketch.js:959: let stackH =  
revH + gap + btnH;  
sketch.js:971: let by = topY  
+ revH + gap;  
sketch.js:1128: let gap = 24;  
sketch.js:1129: let stackH =  
revH + gap + btnH;  
sketch.js:1131: let by = height
```

```
🔍 Search(pattern: "class revH + gap;  
World|updatePlatforms|addPlatform|r  
andom.*platform|platform.*random")
```

```
└ Found 2 lines  
sketch.js:709:  
world.updatePlatforms(player);  
sketch.js:722:
```

```
world.updatePlatforms(player);
```

🔍 Search(pattern: "**/*.js")

└ Found 7 files
blobPlayer.js
libraries/p5.min.js
libraries/p5.sound.min.js
npc.js
platform.js
worldLevel.js
sketch.js

🔍 Search(pattern: "gap|spacing|distance|levelIndex.*2|level.*3")

└ Found 0 lines

🔍 Read(/Users/giangtran/Documents/GitHub/A2_Group14B_FinalGame/worldLevel.js)

└ Read 119 lines

12:24 PM claude-sonnet-4-6

🔍 The platforms come from a JSON file. Let me look at it.

🔍 Search(pattern: "**/*.json")

└ Found 5 files
jsconfig.json
level1.json
level2.json
level3.json
levels.json

🔍 Read(/Users/giangtran/Documents/GitHub/A2_Group14B_FinalGame/level3.json)

└ Read 106 lines

➤ /btw

└ Usage: /btw <your question>

Search(pattern: "class World|updatePlatforms|addPlatform|random.*platform|platform.*random")

└ Found 2 lines
sketch.js:709:

	<pre>world.updatePlatforms(player); sketch.js:722: world.updatePlatforms(player);</pre> 🔍 Search(pa12:24 PM*claude-sonnet-4-6 🔍 The platforms come from a JSON file. Let me look at it.r.js libraries/p5.min.js 🔍 Search(pattern: "**/*.json") └ Found 5 files jsconfig.json level1.jsonjs level2.json level3.json 🔍 Sealevels.json: "gap spacing distance level Index.*2 level.*3") 🔍 Read(/Users/giangtran/Documents/GitHub/A2_Group14B_FinalGame/level3.json) 🔍 Read(/Users/giangtran/Documents/GitHub/A2_Group14B_FinalGame/level3.json) └ Read 106 lines/worldLevel.js)
for level 3, bring the platforms a little bit closer together, right now it's really hard to jump from one to another. just a little bit	I'll reduce spacing from 150 to 120 units. That means recalculating all y positions and scaling down levelHeight accordingly. 🔍 Write(level3.json) └ Added 86 lines, removed 86 lines <pre>1 { 2 "name": "Train", 3 - "levelHeight": 12200, 3 + "levelHeight": 9900, 4 "gravity": 0.45, 5 "jumpV": -16.0, 6 "theme": { ... 10 }, 11 "start": { 12 "x": 720, 13 - "y": 11950, 13 + "y": 9630, 14 "r": 26 15 }, 16 "checkpoints": [17 - { "y": 8900 }, 18 - { "y": 6050 },</pre>

```

19 - { "y": 3200 }
17 + { "y": 7200 },
18 + { "y": 4900 },
19 + { "y": 2600 }
20 ],
21 "platforms": [
22 - { "x": 0, "y": 12100,
    - "w": 4000, "h": 100, "type
    -: "default" },
23 - { "x": 550, "y": 11860,
    - "w": 220, "h": 20, "type
    -: "invisible", "detectRadi
    -us": 150 },
22 + { "x": 0, "y": 9780,
    + "w": 4000, "h": 100, "type"
    +: "default" },
23 + { "x": 550, "y": 9580,
    + "w": 220, "h": 20, "type"
    +: "invisible", "detectRadiu
    +s": 150 },
24
25 - { "x": 250, "y": 11750,
    - "w": 145, "h": 20, "type":
    - "normal" },
26 - { "x": 760, "y": 11600,
    - "w": 210, "h": 20, "type":
    - "invisible", "detectRadius
    -: 150 },
27 - { "x": 510, "y": 11450,
    - "w": 220, "h": 20, "type":
    - "moving", "moveRange":
    -130, "moveSpeed": 1.5 },
28 - { "x": 250, "y": 11300,
    - "w": 105, "h": 20, "type":
    - "normal" },
29 - { "x": 760, "y": 11150,
    - "w": 220, "h": 20, "type":
    - "slippery" },
30 - { "x": 510, "y": 11000,
    - "w": 140, "h": 20, "type":
    - "normal" },
31 - { "x": 250, "y": 10850,
    - "w": 220, "h": 20, "type":
    - "slow" },
32 - { "x": 760, "y": 10700,
    - "w": 110, "h": 20, "type":
    - "invisible", "detectRadius

```

```

-": 150 },
33 - { "x": 510, "y": 10550,
- "w": 220, "h": 20, "type":
- "slippery" },
34 - { "x": 250, "y": 10400,
- "w": 140, "h": 20, "type":
- "normal" },
35 - { "x": 760, "y": 10250,
- "w": 220, "h": 20, "type":
- "invisible", "detectRadius
-": 150 },
36 - { "x": 510, "y": 10100,
- "w": 105, "h": 20, "type":
- "normal" },
37 - { "x": 250, "y": 9950,
- "w": 220, "h": 20, "type":
- "slow" },
38 - { "x": 760, "y": 9800,
- "w": 140, "h": 20, "type":
- "normal" },
39 - { "x": 510, "y": 9650,
- "w": 220, "h": 20, "type":
- "moving", "moveRange":
-150, "moveSpeed": 1.6 },
40 - { "x": 250, "y": 9500,
- "w": 210, "h": 20, "type":
- "slippery" },
41 - { "x": 760, "y": 9350,
- "w": 145, "h": 20, "type":
- "invisible", "detectRadius
-": 150 },
42 - { "x": 510, "y": 9200,
- "w": 210, "h": 20, "type":
- "normal" },
43 - { "x": 250, "y": 9050,
- "w": 145, "h": 20, "type":
- "slow" },
44 - { "x": 760, "y": 8900,
- "w": 210, "h": 20, "type":
- "moving", "moveRange":
-160, "moveSpeed": 1.7 },
45 - { "x": 510, "y": 8750,
- "w": 110, "h": 20, "type":
- "normal" },
46 - { "x": 250, "y": 8600,
- "w": 210, "h": 20, "type":
- "slippery" },

```

	47 - { "x": 760, "y": 8450, - "w": 145, "h": 20, "type": - "normal" }, 48 - { "x": 510, "y": 8300, - "w": 210, "h": 20, "type": - "slow" }, 49 - { "x": 250, "y": 8150, - "w": 145, "h": 20, "type": - "invisible", "detectRadius -": 150 }, 50 - { "x": 760, "y": 8000, - "w": 210, "h": 20, "type": - "moving", "moveRange": -160, "moveSpeed": 1.7 }, 51 - { "x": 510, "y": 7850, - "w": 220, "h": 20, "type": - "slippery" }, 52 - { "x": 250, "y": 7700, - "w": 105, "h": 20, "type": - "normal" }, 53 - { "x": 760, "y": 7550, - "w": 220, "h": 20, "type": - "falling" }, 54 - { "x": 510, "y": 7400, - "w": 140, "h": 20, "type": - "normal" }, 55 - { "x": 250, "y": 7250, - "w": 220, "h": 20, "type": - "slow" }, 56 - { "x": 760, "y": 7100, - "w": 105, "h": 20, "type": - "normal" }, 57 - { "x": 510, "y": 6950, - "w": 220, "h": 20, "type": - "moving", "moveRange": -180, "moveSpeed": 1.8 }, 58 - { "x": 250, "y": 6800, - "w": 110, "h": 20, "type": - "invisible", "detectRadius -": 150 }, 59 - { "x": 760, "y": 6650, - "w": 220, "h": 20, "type": - "slippery" }, 60 - { "x": 510, "y": 6500, - "w": 140, "h": 20, "type": - "normal" }, 61 - { "x": 250, "y": 6350,
--	--

	- "w": 220, "h": 20, "type": - "invisible", "detectRadius - ": 150 }, 62 - { "x": 760, "y": 6200, - "w": 105, "h": 20, "type": - "slow" }, 63 - { "x": 510, "y": 6050, - "w": 220, "h": 20, "type": - "moving", "moveRange": -180, "moveSpeed": 1.9 }, 64 - { "x": 250, "y": 5900, - "w": 140, "h": 20, "type": - "normal" }, 65 - { "x": 760, "y": 5750, - "w": 220, "h": 20, "type": - "slippery" }, 66 - { "x": 510, "y": 5600, - "w": 105, "h": 20, "type": - "normal" }, 67 - { "x": 250, "y": 5450, - "w": 220, "h": 20, "type": - "slow" }, 68 - { "x": 760, "y": 5300, - "w": 110, "h": 20, "type": - "invisible", "detectRadius - ": 150 }, 69 - { "x": 510, "y": 5150, - "w": 220, "h": 20, "type": - "moving", "moveRange": -190, "moveSpeed": 1.9 }, 70 - { "x": 250, "y": 5000, - "w": 210, "h": 20, "type": - "slippery" }, 71 - { "x": 760, "y": 4850, - "w": 145, "h": 20, "type": - "normal" }, 72 - { "x": 510, "y": 4700, - "w": 210, "h": 20, "type": - "falling" }, 73 - { "x": 250, "y": 4550, - "w": 145, "h": 20, "type": - "normal" }, 74 - { "x": 760, "y": 4400, - "w": 210, "h": 20, "type": - "normal" }, 75 - { "x": 510, "y": 4250, - "w": 220, "h": 20, "type":
--	---

	- "moving", "moveRange": -190, "moveSpeed": 2.0 }, 76 - { "x": 250, "y": 4100, - "w": 105, "h": 20, "type": - "slippery" }, 77 - { "x": 760, "y": 3950, - "w": 220, "h": 20, "type": - "invisible", "detectRadius -": 150 }, 78 - { "x": 510, "y": 3800, - "w": 140, "h": 20, "type": - "slow" }, 79 - { "x": 250, "y": 3650, - "w": 220, "h": 20, "type": - "invisible", "detectRadius -": 150 }, 80 - { "x": 760, "y": 3500, - "w": 105, "h": 20, "type": - "normal" }, 81 - { "x": 510, "y": 3350, - "w": 220, "h": 20, "type": - "normal" }, 82 - { "x": 250, "y": 3200, - "w": 210, "h": 20, "type": - "moving", "moveRange": -170, "moveSpeed": 2.0 }, 83 - { "x": 760, "y": 3050, - "w": 145, "h": 20, "type": - "slippery" }, 84 - { "x": 510, "y": 2900, - "w": 210, "h": 20, "type": - "normal" }, 85 - { "x": 250, "y": 2750, - "w": 145, "h": 20, "type": - "slow" }, 86 - { "x": 760, "y": 2600, - "w": 210, "h": 20, "type": - "invisible", "detectRadius -": 150 }, 87 - { "x": 510, "y": 2450, - "w": 220, "h": 20, "type": - "moving", "moveRange": -160, "moveSpeed": 2.1 }, 88 - { "x": 250, "y": 2300, - "w": 105, "h": 20, "type": - "slippery" }, 89 - { "x": 760, "y": 2150,
--	---

- "w": 220, "h": 20, "type":
- "normal" },
- 90 - { "x": 510, "y": 2000,
- "w": 140, "h": 20, "type":
- "slow" },
- 91 - { "x": 250, "y": 1850,
- "w": 220, "h": 20, "type":
- "falling" },
- 92 - { "x": 760, "y": 1700,
- "w": 105, "h": 20, "type":
- "normal" },
- 93 - { "x": 510, "y": 1550,
- "w": 220, "h": 20, "type":
- "invisible", "detectRadius
- ": 150 },
- 94 - { "x": 250, "y": 1400,
- "w": 140, "h": 20, "type":
- "normal" },
- 95 - { "x": 760, "y": 1250,
- "w": 220, "h": 20, "type":
- "slow" },
- 96 - { "x": 510, "y": 1100,
- "w": 210, "h": 20, "type":
- "moving", "moveRange":
- 150, "moveSpeed": 2.1 },
- 97 - { "x": 250, "y": 950,
- "w": 145, "h": 20, "type":
- "invisible", "detectRadius
- ": 150 },
- 98 - { "x": 760, "y": 800,
- "w": 210, "h": 20, "type":
- "slippery" },
- 99 - { "x": 510, "y": 650,
- "w": 110, "h": 20, "type":
- "normal" },
- 100 - { "x": 250, "y": 500,
- "w": 210, "h": 20, "type":
- "slow" },
- 101 - { "x": 760, "y": 350,
- "w": 145, "h": 20, "type":
- "normal" },
- 102 - { "x": 450, "y": 290,
- "w": 220, "h": 20, "type":
- "normal" },
- 103 - { "x": 600, "y": 200,
- "w": 200, "h": 20, "type":
- "finish" }

	<pre>25 + { "x": 250, "y": 9470, +"w": 145, "h": 20, "type": +"normal" }, 26 + { "x": 760, "y": 9350, +"w": 210, "h": 20, "type": +"invisible", "detectRadius" +: 150 }, 27 + { "x": 510, "y": 9230, +"w": 220, "h": 20, "type": +"moving", "moveRange": 1 +30, "moveSpeed": 1.5 }, 28 + { "x": 250, "y": 9110, +"w": 105, "h": 20, "type": +"normal" }, 29 + { "x": 760, "y": 8990, +"w": 220, "h": 20, "type": +"slippery" }, 30 + { "x": 510, "y": 8870, +"w": 140, "h": 20, "type": +"normal" }, 31 + { "x": 250, "y": 8750, +"w": 220, "h": 20, "type": +"slow" }, 32 + { "x": 760, "y": 8630, +"w": 110, "h": 20, "type": +"invisible", "detectRadius" +: 150 }, 33 + { "x": 510, "y": 8510, +"w": 220, "h": 20, "type": +"slippery" }, 34 + { "x": 250, "y": 8390, +"w": 140, "h": 20, "type": +"normal" }, 35 + { "x": 760, "y": 8270, +"w": 220, "h": 20, "type": +"invisible", "detectRadius" +: 150 }, 36 + { "x": 510, "y": 8150, +"w": 105, "h": 20, "type": +"normal" }, 37 + { "x": 250, "y": 8030, +"w": 220, "h": 20, "type": +"slow" }, 38 + { "x": 760, "y": 7910, +"w": 140, "h": 20, "type": +"normal" }, 39 + { "x": 510, "y": 7790,</pre>
--	--

	<pre> +"w": 220, "h": 20, "type": +"moving", "moveRange": 1 +50, "moveSpeed": 1.6 }, 40 + { "x": 250, "y": 7670, +"w": 210, "h": 20, "type": +"slippery" }, 41 + { "x": 760, "y": 7550, +"w": 145, "h": 20, "type": +"invisible", "detectRadius" +: 150 }, 42 + { "x": 510, "y": 7430, +"w": 210, "h": 20, "type": +"normal" }, 43 + { "x": 250, "y": 7310, +"w": 145, "h": 20, "type": +"slow" }, 44 + { "x": 760, "y": 7190, +"w": 210, "h": 20, "type": +"moving", "moveRange": 1 +60, "moveSpeed": 1.7 }, 45 + { "x": 510, "y": 7070, +"w": 110, "h": 20, "type": +"normal" }, 46 + { "x": 250, "y": 6950, +"w": 210, "h": 20, "type": +"slippery" }, 47 + { "x": 760, "y": 6830, +"w": 145, "h": 20, "type": +"normal" }, 48 + { "x": 510, "y": 6710, +"w": 210, "h": 20, "type": +"slow" }, 49 + { "x": 250, "y": 6590, +"w": 145, "h": 20, "type": +"invisible", "detectRadius" +: 150 }, 50 + { "x": 760, "y": 6470, +"w": 210, "h": 20, "type": +"moving", "moveRange": 1 +60, "moveSpeed": 1.7 }, 51 + { "x": 510, "y": 6350, +"w": 220, "h": 20, "type": +"slippery" }, 52 + { "x": 250, "y": 6230, +"w": 105, "h": 20, "type": +"normal" }, 53 + { "x": 760, "y": 6110, </pre>
--	--

```

+"w": 220, "h": 20, "type":
+"falling" },
54 + { "x": 510, "y": 5990,
+"w": 140, "h": 20, "type":
+"normal" },
55 + { "x": 250, "y": 5870,
+"w": 220, "h": 20, "type":
+"slow" },
56 + { "x": 760, "y": 5750,
+"w": 105, "h": 20, "type":
+"normal" },
57 + { "x": 510, "y": 5630,
+"w": 220, "h": 20, "type":
+"moving", "moveRange": 1
+80, "moveSpeed": 1.8 },
58 + { "x": 250, "y": 5510,
+"w": 110, "h": 20, "type":
+"invisible", "detectRadius"
+: 150 },
59 + { "x": 760, "y": 5390,
+"w": 220, "h": 20, "type":
+"slippery" },
60 + { "x": 510, "y": 5270,
+"w": 140, "h": 20, "type":
+"normal" },
61 + { "x": 250, "y": 5150,
+"w": 220, "h": 20, "type":
+"invisible", "detectRadius"
+: 150 },
62 + { "x": 760, "y": 5030,
+"w": 105, "h": 20, "type":
+"slow" },
63 + { "x": 510, "y": 4910,
+"w": 220, "h": 20, "type":
+"moving", "moveRange": 1
+80, "moveSpeed": 1.9 },
64 + { "x": 250, "y": 4790,
+"w": 140, "h": 20, "type":
+"normal" },
65 + { "x": 760, "y": 4670,
+"w": 220, "h": 20, "type":
+"slippery" },
66 + { "x": 510, "y": 4550,
+"w": 105, "h": 20, "type":
+"normal" },
67 + { "x": 250, "y": 4430,
+"w": 220, "h": 20, "type":

```

	<pre> +"slow" }, 68 + { "x": 760, "y": 4310, +"w": 110, "h": 20, "type": +"invisible", "detectRadius" +: 150 }, 69 + { "x": 510, "y": 4190, +"w": 220, "h": 20, "type": +"moving", "moveRange": 1 +90, "moveSpeed": 1.9 }, 70 + { "x": 250, "y": 4070, +"w": 210, "h": 20, "type": +"slippery" }, 71 + { "x": 760, "y": 3950, +"w": 145, "h": 20, "type": +"normal" }, 72 + { "x": 510, "y": 3830, +"w": 210, "h": 20, "type": +"falling" }, 73 + { "x": 250, "y": 3710, +"w": 145, "h": 20, "type": +"normal" }, 74 + { "x": 760, "y": 3590, +"w": 210, "h": 20, "type": +"normal" }, 75 + { "x": 510, "y": 3470, +"w": 220, "h": 20, "type": +"moving", "moveRange": 1 +90, "moveSpeed": 2.0 }, 76 + { "x": 250, "y": 3350, +"w": 105, "h": 20, "type": +"slippery" }, 77 + { "x": 760, "y": 3230, +"w": 220, "h": 20, "type": +"invisible", "detectRadius" +: 150 }, 78 + { "x": 510, "y": 3110, +"w": 140, "h": 20, "type": +"slow" }, 79 + { "x": 250, "y": 2990, +"w": 220, "h": 20, "type": +"invisible", "detectRadius" +: 150 }, 80 + { "x": 760, "y": 2870, +"w": 105, "h": 20, "type": +"normal" }, 81 + { "x": 510, "y": 2750, +"w": 220, "h": 20, "type": </pre>
--	---

	<pre> +"normal" }, 82 + { "x": 250, "y": 2630, +"w": 210, "h": 20, "type": +"moving", "moveRange": 1 +70, "moveSpeed": 2.0 }, 83 + { "x": 760, "y": 2510, +"w": 145, "h": 20, "type": +"slippery" }, 84 + { "x": 510, "y": 2390, +"w": 210, "h": 20, "type": +"normal" }, 85 + { "x": 250, "y": 2270, +"w": 145, "h": 20, "type": +"slow" }, 86 + { "x": 760, "y": 2150, +"w": 210, "h": 20, "type": +"invisible", "detectRadius" +: 150 }, 87 + { "x": 510, "y": 2030, +"w": 220, "h": 20, "type": +"moving", "moveRange": 1 +60, "moveSpeed": 2.1 }, 88 + { "x": 250, "y": 1910, +"w": 105, "h": 20, "type": +"slippery" }, 89 + { "x": 760, "y": 1790, +"w": 220, "h": 20, "type": +"normal" }, 90 + { "x": 510, "y": 1670, +"w": 140, "h": 20, "type": +"slow" }, 91 + { "x": 250, "y": 1550, +"w": 220, "h": 20, "type": +"falling" }, 92 + { "x": 760, "y": 1430, +"w": 105, "h": 20, "type": +"normal" }, 93 + { "x": 510, "y": 1310, +"w": 220, "h": 20, "type": +"invisible", "detectRadius" +: 150 }, 94 + { "x": 250, "y": 1190, +"w": 140, "h": 20, "type": +"normal" }, 95 + { "x": 760, "y": 1070, +"w": 220, "h": 20, "type": +"slow" }, </pre>
--	--

	<pre> 96 + { "x": 510, "y": 950, + "w": 210, "h": 20, "type": + "moving", "moveRange": 1 + 50, "moveSpeed": 2.1 }, 97 + { "x": 250, "y": 830, + "w": 145, "h": 20, "type": + "invisible", "detectRadius" +: 150 }, 98 + { "x": 760, "y": 710, + "w": 210, "h": 20, "type": + "slippery" }, 99 + { "x": 510, "y": 590, + "w": 110, "h": 20, "type": + "normal" }, 100 + { "x": 250, "y": 470, + "w": 210, "h": 20, "type": + "slow" }, 101 + { "x": 760, "y": 350, + "w": 145, "h": 20, "type": + "normal" }, 102 + { "x": 450, "y": 290, + "w": 220, "h": 20, "type": + "normal" }, 103 + { "x": 600, "y": 200, + "w": 200, "h": 20, "type": + "finish" } 104] 105 } </pre>  Done. Spacing reduced from 150 → 120 units. The level height and all positions have been recalculated accordingly (levelHeight: 9900, start: y=9630). Checkpoints updated to y=7200, 4900, 2600 to stay evenly spread.
make some more platforms of level 3 shorter, about 1/2 of their current length	Looking at the current file, many platforms are still at 210–220. I'll reduce roughly half of those remaining full-size ones down to ~105–110. Searched for 1 pattern (ctrl+o to expand)  I'll reduce about half of the

remaining full-size (210–220) platforms. I'll focus on normal/slippy/slow ones and leave moving/falling at full size.

● Update(level3.json)

└ Added 1 line, removed 1 line

```
44     "moveSpeed": 1.5
45   },
46   { "x": 250, "y": 9110, "
      w": 105, "h": 20, "type": "n
      ormal" },
47 - { "x": 760, "y": 8990, "
      -w": 220, "h": 20, "type": "s
      -lippy" },
47 + { "x": 760, "y": 8990, "
      +w": 110, "h": 20, "type": "s
      +lippy" },
48   { "x": 510, "y": 8870, "
      w": 140, "h": 20, "type": "n
      ormal" },
49   { "x": 250, "y": 8750, "
      w": 220, "h": 20, "type": "s
      low" },
50   {
```

● Update(level3.json)

└ Added 1 line, removed 1 line

```
55     "type": "invisible",
56     "detectRadius": 150
57   },
58 - { "x": 510, "y": 8510, "
      -w": 220, "h": 20, "type": "s
      -lippy" },
58 + { "x": 510, "y": 8510, "
      +w": 110, "h": 20, "type": "s
      +lippy" },
59   { "x": 250, "y": 8390, "
      w": 140, "h": 20, "type": "n
      ormal" },
```

```
60  {
61    "x": 760,
```

Update(level3.json)

└ Added 1 line, removed 1 line

```
77    "moveRange": 150,
78    "moveSpeed": 1.6
79  },
80 -  { "x": 250, "y": 7670, "
    -w": 210, "h": 20, "type": "s
    -lippery" },
80 +  { "x": 250, "y": 7670, "
    +w": 105, "h": 20, "type": "s
    +lippery" },
81  {
82    "x": 760,
83    "y": 7550,
```

Update(level3.json)

└ Added 1 line, removed 1 line

```
86    "type": "invisible",
87    "detectRadius": 150
88  },
89 -  { "x": 510, "y": 7430, "
    -w": 210, "h": 20, "type": "n
    -ormal" },
89 +  { "x": 510, "y": 7430, "
    +w": 105, "h": 20, "type": "n
    +ormal" },
90  { "x": 250, "y": 7310, "
    w": 145, "h": 20, "type": "s
    low" },
91  {
92    "x": 760,
```

Update(level3.json)

└ Added 1 line, removed 1 line

```
98     "moveSpeed": 1.7
99   },
100   { "x": 510, "y": 7070,
      "w": 110, "h": 20, "type":
      "normal" },
101 -   { "x": 250, "y": 6950,
      -"w": 210, "h": 20, "type":
      -"slippery" },
101 +   { "x": 250, "y": 6950,
      +"w": 105, "h": 20, "type":
      +"slippery" },
102   { "x": 760, "y": 6830,
      "w": 145, "h": 20, "type":
      "normal" },
103   { "x": 510, "y": 6710,
      "w": 210, "h": 20, "type":
      "slow" },
104   {
```

Update(level3.json)

└ Added 1 line, removed 1 line

```
118     "moveRange": 160,
119     "moveSpeed": 1.7
120   },
121 -   { "x": 510, "y": 6350,
      -"w": 220, "h": 20, "type":
      -"slippery" },
121 +   { "x": 510, "y": 6350,
      +"w": 110, "h": 20, "type":
      +"slippery" },
122   { "x": 250, "y": 6230,
      "w": 105, "h": 20, "type":
      "normal" },
```

	<pre> 123 { "x": 760, "y": 6110, "w": 220, "h": 20, "type": "falling" }, 124 { "x": 510, "y": 5990, "w": 140, "h": 20, "type": "normal" }, </pre>  Update(level3.json) Added 1 line, removed 1 line <pre> 122 { "x": 250, "y": 6230, "w": 105, "h": 20, "type": "normal" }, 123 { "x": 760, "y": 6110, "w": 220, "h": 20, "type": "falling" }, 124 { "x": 510, "y": 5990, "w": 140, "h": 20, "type": "normal" }, 125 - { "x": 250, "y": 5870, -"w": 220, "h": 20, "type": -"slow" }, 125 + { "x": 250, "y": 5870, +"w": 110, "h": 20, "type": +"slow" }, 126 { "x": 760, "y": 5750, "w": 105, "h": 20, "type": "normal" }, 127 { 128 "x": 510, </pre>  Update(level3.json)
make the jump heights a bit stronger for level 3 right now i cant even jump onto the first platform	Let me check exactly why — with jumpV: -14 and gravity: 0.45, max jump height is ~218 units. The floor is at y=12100, the first invisible platform is at y=11860 — that's a 240-unit gap, which exceeds the max jump. I just need to increase jumpV for level 3.  Update(level3.json) Added 1 line, removed 1 line <pre> 2 "name": "Train", 3 "levelHeight": 12200, 4 "gravity": 0.45, </pre>

	<pre> 5 - "jumpV": -14.0, 5 + "jumpV": -16.0, 6 "theme": { 7 "bg": "#E8EEF5", 8 "platform": "#7A9BB5", </pre>  jumpV: -16 raises the max jump height from ~218 to ~284 units, which clears the 240-unit gap to the first platform with comfortable margin — and still requires deliberate jumping for the 150-unit gaps between platforms throughout the level. * Cogitated for 45s
https://github.com/k75chen/A2_Group14B_MidTermGame.git help me edit level 2.json	Let me fetch the repository to understand the project structure. Unfortunately, I'm not able to directly access GitHub repositories through my tools. Here are a few ways we can work together on your <code>level2.json</code>: Option 1 — Upload the file here Copy the <code>level2.json</code> file from your repo and upload it directly to this chat. I can then edit it and give you the updated version back. Option 2 — Paste the contents Open <code>level2.json</code> in a text editor, copy the contents, and paste them here. I'll make the edits and return the updated JSON. Once I can see the file, just let me know what changes you'd like — adding enemies, adjusting platforms, changing spawn points, etc. — and I'll get it done!
<pre>{ "name": "House",</pre>	Looking at level 1's structure, I'll build out level 2 with the same density and checkpoint progression, but with train-themed platform types. Let me design a full platform layout for the 8400 height.

<pre> "levelHeight": 6000, "gravity": 0.2, "jumpV": -16.0, "theme": { "bg": "#FFF5E6", "platform": "#D4956A", "blob": "#5B4FCF" }, "start": { "x": 720, "y": 5800, "r": 26 }, "checkpoints": [{ "y": 4100 }, { "y": 2720 }, { "y": 1480 }], "platforms": [{ "x": 0, "y": 5900, "w": 4000, "h": 100, "type": "default" }, { "x": 550, "y": 5740, "w": 200, "h": 20, "type": "table" }, { "x": 900, "y": 5580, "w": 180, "h": 20, "type": "piano" }, { "x": 200, "y": 5430, "w": 220, "h": 20, "type": "couch" }, { "x": 1100, "y": 5270, "w": 160, "h": 20, "type": "chair" }, { "x": 650, "y": 5120, "w": 200, "h": 20, "type": "bathtub" }, { "x": 300, "y": 4970, "w": 220, "h": 20, "type": "beanbag" }, { "x": 1000, "y": 4820, "w": 180, "h": 20, "type": "tv" }, { "x": 500, "y": 4680, "w": 200, "h": 20, "type": "bed" }, { "x": 150, "y": 4530, "w": </pre>	Created a file, read a file Created a file, read a file Here's your updated <code>level12.json</code>! Here's a summary of what was added: Platform types used (all train-themed): • <code>bench</code> — standard train bench seating • <code>seat</code> — individual passenger seat • <code>luggage_rack</code> — overhead storage shelf • <code>dining_table</code> — dining car table • <code>conductor_stand</code> — conductor's podium/desk Structure mirrors level 1: • 47 platforms total across 8400 height (same density as level 1's 37 across 6000) • Platforms alternate left/right (x: 100–1100) so the player zigzags upward • Vertical gaps of ~160–200px between platforms, consistent with level 1's jumpability • Checkpoints at y: 5600, 3600, and 1800 are all reachable You'll just need to make sure your renderer handles the new type strings (e.g. draws <code>bench</code> to look like a train bench). Let me know if you want to tweak gaps, add hazards, or adjust anything!
---	---

190, "h": 20, "type": "table" }, { "x": 900, "y": 4380, "w": 160, "h": 20, "type": "piano" }, { "x": 600, "y": 4240, "w": 180, "h": 20, "type": "couch" }, { "x": 250, "y": 4100, "w": 200, "h": 20, "type": "chair" }, { "x": 1050, "y": 3840, "w": 170, "h": 20, "type": "bathtub" }, { "x": 400, "y": 3680, "w": 200, "h": 20, "type": "beanbag" }, { "x": 750, "y": 3520, "w": 160, "h": 20, "type": "tv" }, { "x": 150, "y": 3360, "w": 210, "h": 20, "type": "bed" }, { "x": 950, "y": 3200, "w": 180, "h": 20, "type": "table" }, { "x": 500, "y": 3040, "w": 160, "h": 20, "type": "piano" }, { "x": 200, "y": 2880, "w": 220, "h": 20, "type": "couch" }, { "x": 850, "y": 2720, "w": 170, "h": 20, "type": "chair" }, { "x": 450, "y": 2560, "w": 190, "h": 20, "type": "bathtub" }, { "x": 100, "y": 2440, "w": 180, "h": 20, "type": "beanbag" }, { "x": 900, "y": 2280, "w": 160, "h": 20, "type": "tv" }, { "x": 550, "y": 2120, "w":	
---	--


```
200, "h": 20, "type": "bed" },  
  
  { "x": 250, "y": 1960, "w":  
180, "h": 20, "type": "table" },  
  
  { "x": 1000, "y": 1800, "w":  
160, "h": 20, "type": "piano" },  
  
  { "x": 450, "y": 1640, "w":  
200, "h": 20, "type": "couch" },  
  
  { "x": 150, "y": 1480, "w":  
220, "h": 20, "type": "chair" },  
  
  { "x": 800, "y": 1380, "w":  
170, "h": 20, "type": "bathtub"  
},  
  
  { "x": 500, "y": 1140, "w":  
190, "h": 20, "type":  
"beanbag" },  
  
  { "x": 200, "y": 990, "w":  
200, "h": 20, "type": "tv" },  
  
  { "x": 900, "y": 830, "w":  
180, "h": 20, "type": "bed" },  
  
  { "x": 450, "y": 670, "w":  
200, "h": 20, "type": "table" },  
  
  { "x": 100, "y": 510, "w":  
160, "h": 20, "type": "piano" },  
  
  { "x": 750, "y": 360, "w":  
200, "h": 20, "type": "couch" },  
  
  { "x": 400, "y": 220, "w":  
180, "h": 20, "type": "chair" },  
  
  { "x": 600, "y": 80, "w": 200,  
"h": 20, "type": "finish" }  
  
]  
}
```

this is level 1.

```
{  
  "name": "Train",  
  "levelHeight": 8400,  
  "gravity": 0.22,  
  "jumpV": -16.0,  
  "theme": { "bg": "#E8EEF5",  
    "platform": "#7A9BB5",  
    "blob": "#5B4FCF" },  
  "start": { "x": 720, "y": 8200,  
    "r": 26 },  
  "checkpoints": [{ "y": 5600 },  
    { "y": 3600 }, { "y": 1800 }],  
  "platforms": [  
    { "x": 0, "y": 8300, "w":  
4000, "h": 100, "type":  
"default" },  
    { "x": 600, "y": 8100, "w":  
200, "h": 20, "type": "default"  
},  
    { "x": 600, "y": 200, "w":  
200, "h": 20, "type": "finish" }  
  ]  
}
```

this is level 2 so far. we are building a platform game and i want the layout to be similar to level 1, but the platforms must be "train themed." I want the platforms to look like train benches

It sometimes resets in the middle of the level and takes you back to the beginning, which should not happen	Looking at your level data, the issue is likely with the checkpoints. In level 1 you have 3 checkpoints, and level 2 also has 3 — but the checkpoint at y: 5600 is positioned between platforms but may not be close enough to any platform for the player to actually trigger it before dying and respawning from the start. Can you share a bit more about how checkpoints work in your code? Specifically: Also, if you can share the relevant game logic code (the file that handles checkpoints, player death, and respawning), I can pinpoint the exact bug rather than guessing!
/* Platform.js (Example 5) A Platform is a single axis-aligned rectangle in the world. Why a class for something "simple"? - It standardizes the shape of platform data. - It makes later upgrades easy (e.g., moving platforms, icy platforms, spikes). - It keeps drawing code in the object that knows what it is.	I can see the issue! Your Platform.js only has drawing cases for the level 1 types (bed, table, piano, couch, etc.). The new train-themed types (bench, seat, luggage_rack, dining_table, conductor_stand) fall through to the default case — that part is fine for drawing. But the reset bug is almost certainly caused by the gaps between platforms being too large for the jump physics. Looking at level 2: • levelHeight: 8400 vs level 1's 6000 — that's 40% taller • But jumpV is still -16.0 and gravity is 0.22 (vs 0.2 in level 1) The higher gravity + same jump velocity means the player can't reach platforms that are spaced the same as level 1. When they fall past a checkpoint without triggering it, they die and reset to the start. Let me fix the platform spacing and also add the train drawing cases: Created 2 files, ran a command Created 2 files, ran a command

In JSON, platforms are stored like:

```
{ "x": 0, "y": 324, "w": 640, "h": 36 }
```

```
*/
```

```
class Platform {
```

```
  constructor({ x, y, w, h, type }) {
```

```
    // Position is the top-left corner.
```

```
    this.x = x;
```

```
    this.y = y;
```

```
    // Size (width/height).
```

```
    this.w = w;
```

```
    this.h = h;
```

```
    // Semantic type used for drawing.
```

```
    this.type = type || "default";
```

```
  }
```

```
  draw(fillColor) {
```

```
    if (this.type === "checkpoint") return; // never draw checkpoints
```

Now let me also give you the **Platform.js** with the train drawing cases added:

Created a file, read a file

Created a file, read a file

Two files updated. Here's what was fixed and why:

The reset bug — platform gaps were too large. With **gravity: 0.22** (vs level 1's **0.2**) and the same **jumpV: -16.0**, the max jump height is slightly shorter. The previous version had gaps of ~160–200px which worked for level 1 but caused the player to fall past checkpoints without triggering them, resetting to start. All gaps are now capped at **~180px** to guarantee every jump is reachable.

Also included — train draw cases in **Platform.js:**

- **bench** — two-cushion bench seat in steel blue
- **seat** — single cushion with a small backrest nub
- **luggage_rack** — slatted metal rack with side bars
- **dining_table** — fold-down tray table with centre fold line and legs
- **conductor_stand** — dark podium with a gold stripe

```
noStroke();

switch (this.type) {

  case "bed":

    fill("#F4A58A");

    rect(this.x, this.y, this.w,
this.h);

    fill("#E8907A");

    rect(this.x + 6, this.y - 8,
this.w * 0.3, 10); // left pillow

    rect(this.x + this.w *
0.6, this.y - 8, this.w * 0.3,
10); // right pillow

    break;

  case "table":

    fill("#8B5E3C");

    rect(this.x, this.y, this.w,
this.h);

    fill("#6B4423");

    rect(this.x + 8, this.y +
this.h, 10, 14); // left leg

    rect(this.x + this.w - 18,
this.y + this.h, 10, 14); //
right leg

    break;

  case "piano":

    fill("#2C2C2C");

    rect(this.x, this.y, this.w,
this.h);

    fill("#FFFFFF");

    stroke("#AAAAAA");
```

```
strokeWeight(1);

let keyW = this.w / 9;

for (let k = 0; k < 8; k++)
{
    rect(this.x + k * keyW +
2, this.y, keyW - 2, this.h *
0.8);

    }

    noStroke();

    break;

case "couch":

    fill("#C4876B");

    rect(this.x, this.y, this.w,
this.h);

    fill("#B07558");

    rect(this.x, this.y - 10,
16, this.h + 10); // left arm

    rect(this.x + this.w - 16,
this.y - 10, 16, this.h + 10); //
right arm

    break;

case "chair":

    fill("#C9956C");

    rect(this.x, this.y, this.w,
this.h);

    break;

case "bathtub":

    fill("#A8D8EA");

    rect(this.x, this.y, this.w,
this.h);
```

```
fill("#8EC8DA");

    arc(this.x + 20, this.y +
this.h / 2, 28, 28, PI / 2, PI *
1.5); // faucet end

    break;

    case "beanbag":

        fill("#9B89B5");

        ellipse(this.x + this.w /
2, this.y + this.h / 2, this.w,
this.h * 3);

        break;

    case "tv":

        fill("#3A3A3A");

        rect(this.x, this.y, this.w,
this.h);

        fill("#111111");

        rect(this.x + 6, this.y +
3, this.w - 12, this.h - 6); //
screen

        break;

    case "finish":

        let pulse =
sin(frameCount * 0.05) * 4;

        fill("#FFD700");

        rect(

            this.x - pulse / 2,

            this.y - pulse / 2,

            this.w + pulse,

            this.h + pulse,
```

<pre>); break; default: fill(fillColor); rect(this.x, this.y, this.w, this.h); } } } </pre>	
Instead of editing platform.js, can you make a level 2 platform file that only applies to level 2?	Looking at your code, the cleanest way to do this is to create a <code>Platform2.js</code> that extends the base <code>Platform</code> class, adding the train cases and falling back to the parent for everything else. Ran a command, created a file, read a file Ran a command, created a file, read a file <code>Platform2.js</code> extends <code>Platform</code> and only overrides the train-specific types. To wire it up, you just need one small change in your level-loading code — swap the class based on which level is active: <pre> javascript const PlatformClass = currentLevel === 2 ? Platform2 : Platform; platforms = levelData.platforms.map(p => new PlatformClass(p)); </pre> Make sure <code>Platform.js</code> is loaded before <code>Platform2.js</code> in your HTML so the parent class exists when the child is defined.

Actually, add the fixed code to platform.js	All 5 train cases (bench, seat, luggage_rack, dining_table, conductor_stand) are now added directly to Platform.js under a // Level 2: Train comment section, keeping everything organized alongside the existing level 1 types.
It still sometimes resets on the first level, and there is also a level 3 but it is ending at level 2. <pre>{ "name": "Office", "levelHeight": 10800, "gravity": 0.25, "jumpV": -16.0, "theme": { "bg": "#F0EDE8", "platform": "#9A8E82", "blob": "#5B4FCF" }, "start": { "x": 720, "y": 10600, "r": 26 }, "checkpoints": [{ "y": 7200 }, { "y": 4600 }, { "y": 2300 }], "platforms": [{ "x": 0, "y": 10700, "w": 4000, "h": 100, "type": "default" }, { "x": 600, "y": 10500, "w": 200, "h": 20, "type": "default" }, { "x": 600, "y": 200, "w": 200, "h": 20, "type": "finish" }] }</pre>	Two separate issues here. Let me tackle both: 1. Level 1 reset bug — I need to actually calculate the max jump height from the physics to verify the gaps are safe 2. Level 3 — same platform-filling treatment as level 2, plus office-themed types added to Platform.js Good — now I have exact safe gap limits. I'll use ~140px max gap for level 1 (giving comfortable margin below 648px), ~160px for level 2, and ~150px for level 3. Let me also check level 1's current gaps: Find all large gaps in level 1 Find all large gaps in level 1 There are two problem gaps in level 1 — 260px and 240px. I'll fix those and fill out level 3 now. Created 2 files, ran a command Created 2 files, ran a command Clean outputs Script Fixed level1.json with problem gaps patched level1.json level3.json fully filled out with office theme level3.json Done Now Platform.js with office types added:

<pre>}</pre>	Created a file, read a file Created a file, read a file Here's what was done across all three files: Level 1 fix — the physics simulation revealed two gaps that were too large: a 260px gap between y:4100 and y:3840, and a 240px gap between y:1380 and y:1140. Both had a bridging platform inserted to bring every gap under 160px (the max safe jump height with gravity: 0.2 is 648px, but you need comfortable margin for horizontal movement). Level 3 — fully filled from y:10700 down to y:80 with ~150px gaps (safe for gravity: 0.25, max jump height 520px). Platform types used: desk, cabinet, shelf, and printer. Platform.js — four office draw cases added: a desk with drawer block and legs, a filing cabinet with handles, a colour-coded bookshelf, and a printer with a paper slot and green indicator light.
Level 3 does not need any platforms added for now, it just needs to open to level 3 after completing level 2	level 3 after completing level 2 Mar 4 That's a logic issue in your game code, not the JSON. Can you share the file that handles level completion and progression? It's likely something like Game.js, Sketch.js, or main.js — wherever you check if the player hits the finish platform and loads the next level. The logic is already there — the "win" state triggers when the player lands on finish, and pressing ENTER calls loadLevel(levelIndex + 1). So the issue is simpler: totalLevels is set to 3 but the win condition checks levelIndex >= totalLevels - 1, meaning it treats level index 1 (level 2) as the last level and shows "complete" instead of "win". Wait — actually totalLevels = 3 and levelIndex

starts at 0, so level 2 is index 1 and level 3 is index 2. That math is correct. The real problem is here:

javascript

```
let climbProgress = map(player.y, 5800, 80, 0, 1);
```

This is **hardcoded to level 1's coordinates** (`y: 5800` start, `y: 80` top). On level 2 and 3 the player starts much lower, so `climbProgress` goes way out of range and stress spikes instantly — but more critically, the **fall threshold respawn logic** uses `activeCheckpointY` which initializes to `world.start.y`. If a checkpoint is never activated before a fall, it respawns at the start correctly. The bug is actually in the checkpoint activation check:

javascript

```
if (player.y < cy && cy < activeCheckpointY) {  
    activeCheckpointY = cy;  
}
```

This only activates a checkpoint if `cy < activeCheckpointY` — meaning it only triggers checkpoints **above** the current one. But `activeCheckpointY` starts at `world.start.y` (e.g. 8200 for level 2). The checkpoints are at 5600, 3600, 1800 — all lower numbers (higher up). So `cy < activeCheckpointY` is always true initially... that part is fine.

The actual reset-to-beginning bug is the **`climbProgress` map being hardcoded**. Here's the fix:

Created a file, read a file

Created a file, read a file

Only one line changed — the `climbProgress` map in the stress section:

Before:

javascript

	<pre>let climbProgress = map(player.y, 5800, 80, 0, 1);</pre> After: javascript <pre>let levelStartY = world.start.y;</pre> <pre>let levelTopY = 80;</pre> <pre>let climbProgress = map(player.y, levelStartY, levelTopY, 0, 1);</pre> The old code hardcoded level 1's start Y (5800). On level 2 the player starts at 8200, so climbProgress immediately calculated as way above 1.0, pinning stress at 100 from the very first frame. That caused a cascade of falls and resets before the player could do anything. Now it reads the start Y dynamically from whichever level is loaded.
Vscode agent said yyou need to set up GitHub Copilot and be signed in to use Chat.	That message is from VS Code — it's nothing to do with me! I'm Claude, running in your browser at claude.ai, completely separate from VS Code and GitHub Copilot. You can keep working with me here by just pasting your code directly into the chat. To get your files into your project, simply download them from the chat and drop them into your project folder in VS Code — no Copilot needed.

Keyan: combine these two images with a glitch effect to make them look like they are fading into eachother.

Reve: I'll create a glitchy blend of these two mood boards — the warm retro-tech aesthetic merging with the dreamy blue sky aesthetic. Let me generate a few variations.

Keyan: add a bit more vhs retro feel. i dont love the very whipey and blurry, but i like the distorition. have the outer sides of the imagebe abit more clear (constrain distrotrion to middle a bit more

Reve: Let me refine this — keeping the distortion concentrated in the middle transition zone while making the edges clearer and adding more VHS texture throughout.

Keyan: can you use resolution 1200x10800

Reve: That's a very tall resolution (1200×10800, roughly a 1:9 ratio). I can generate at the closest supported aspect ratio of 9:16, but the exact pixel dimensions I can produce are limited by the generation tools. Let me create it at the tallest aspect ratio available and then upscale it for you to get closer to that resolution.

Let me generate a few versions and upscale the best one

Keyan: Generate separate top, middle, and bottom sections to stitch

Reve: The maximum native generation size is 768×1360 at 9:16. Unfortunately, the exact 1200×10800 ratio (roughly 1:9) isn't a supported aspect ratio — the tallest available is 9:16.

I can upscale one of these to get you a higher resolution version. Would you like me to upscale one of these, or were you looking for something much taller in proportion (like a scrollable image)? If you need a true 1:9 ratio, one approach would be to generate multiple sections and stitch them together outside of Reve.

I'll generate three separate sections that you can stitch together into one tall 1200×10800 image — each section will share the same aesthetic and flow seamlessly into the next.

Keyan: top section looks really good, but the other look a bit off style wise. they wont stick into eachtoehr well. can you do the same to the middle and bottom so that its consinant in glitch style? you can reference the current top along with the attatched version

Reve: I'll regenerate the middle and bottom sections to match the style of the top section, using both the active image and the uploaded reference for consistency.

Here are your options — two middle sections and two bottom sections, all styled to match the top:

Middle sections:

Middle Section v2 — heavy glitch with more horizontal displacement, very intense collision zone

Middle Section v2 Alt — similar intensity with the anime CRT monitor more visible, strong horizontal scan line artifacts

Bottom sections:

Bottom Section v2 — cool blue dominant with goldfish bowl, jellyfish, clouds, rainbow, and fading warm elements on the left

Bottom Section v2 Alt — similar dreamcore ending with goldfish, jellyfish, pixel stars, and warm sunset remnants fading out

Pick your preferred middle and bottom to pair with the top section you already downloaded.

Keyan: generate new versions for middle section. currently they dont match and are much too glitchy

Reve: I'll tone down the glitch intensity and match the style of the top section more closely.

These are much more consistent with the top section now. Middle Section v3 and v3 Alt closely mirror the top's composition and glitch intensity. Middle Section v3 Edit introduces a goldfish bowl in the lower right as a transition element, and v3 C has both a goldfish bowl and rainbow appearing on the right side — nice for bridging into the bottom section.